

Programming Paradigm

Introduction

Ashish Kr. Layek (CST, IEST)

Computer Program

- ❑ A computer program (or just a program or software) is a sequence of instructions, written to perform a specified task with a computer.
 - ❑ A program is expected to behave in a predetermined manner. No matter how many times one program is run, the same result should be received for same set of data provided
-

Programming Language

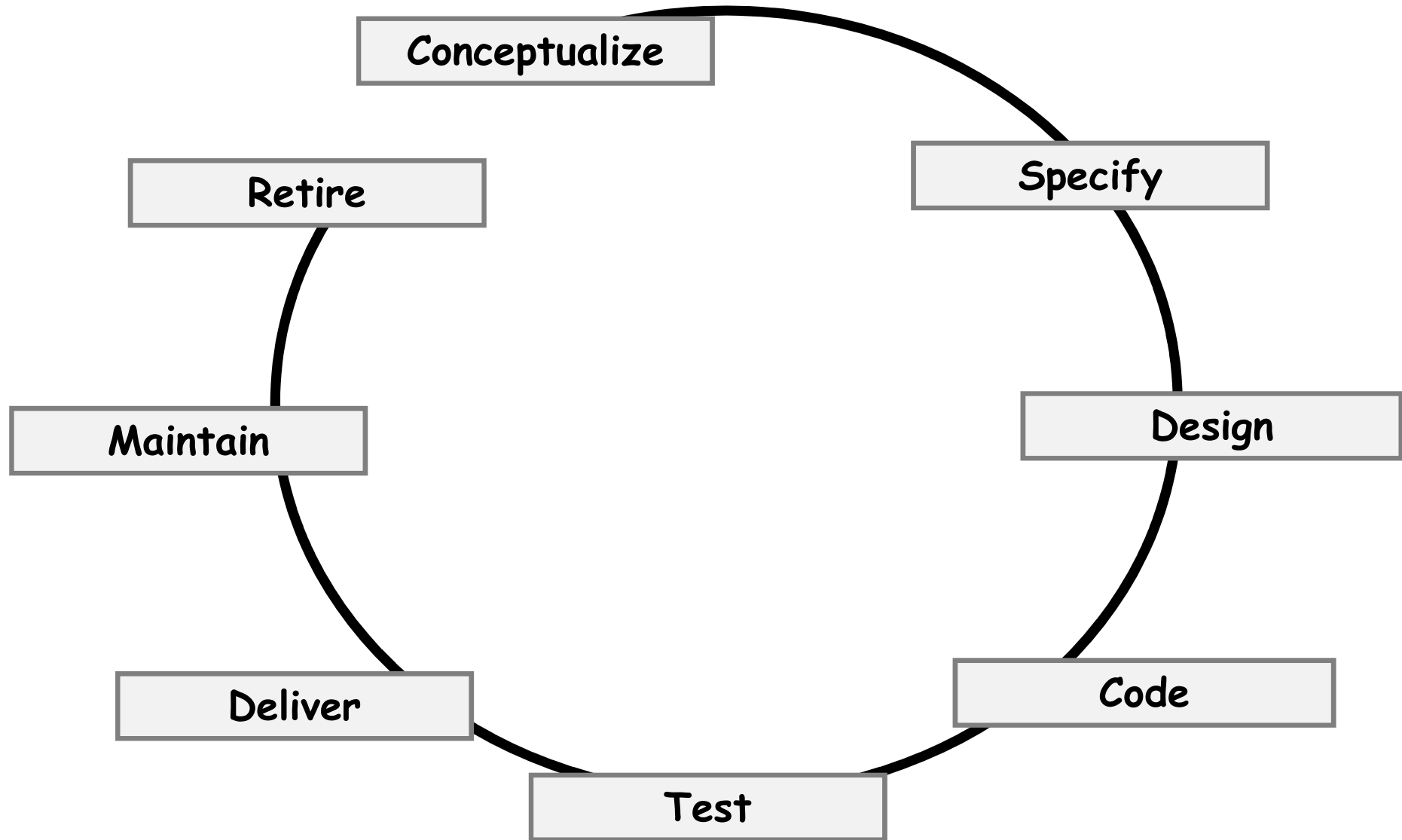
- ❑ Machine Language – Strict binary form / byte code
- ❑ High Level Language – C, Cobol, C++, Java, LISP etc

Note : High level languages are compiled or interpreted to Machine language before execution

Computer Programming

- ❑ Computer programming is the iterative process of writing or editing source code that can be executed in a computer
- ❑ It involves testing, analyzing, refining, and sometimes coordinating with other programmers on a jointly developed program

Life cycle of a Software



Programming Language Timeline

- **FlowMatic** - 1955 Grace Hopper UNIVAC
 - **ForTran** - 1956 John Backus IBM
 - **AlgOL** - 1958 ACM Language Committee
 - **LISP** - 1958 John McCarthy MIT
 - **CoBOL** - 1960 Committee on Data Systems Languages
 - **BASIC** - 1964 John Kemeny & Thomas Kurtz Dartmouth
 - **PL/I** - 1964 IBM Committee
 - **Simula** - 1967 Norwegian Computing Center Kristen Nygaard & Ole-Johan Dahl
 - **Logo** - 1968 Seymour Papert MIT
 - **Pascal** - 1970 Nicklaus Wirth Switzerland
 - **C** - 1972 Dennis Ritchie & Kenneth Thompson Bell Labs
 - **Smalltalk** - 1972 Alan Kay Xerox PARC
 - **ADA** - 1981 DOD
 - **Objective C** - 1985 Brad Cox Stepstone Systems
 - **C++** - 1986 Bjarne Stroustrup Bell Labs
 - **Eiffel** - 1989 Bertrand Meyer France
 - **Visual BASIC** - 1990 Microsoft
 - **Delphi** - 1995 Borland
 - **Object CoBOL** - 1995 MicroFocus
 - **Java** - 1995 Sun Microsystems
-

Five Generations of Programming Languages

- ❑ First - Machine Languages
 - machine codes
- ❑ Second - Assembly Languages
 - symbolic assemblers
- ❑ Third - High Level Procedural Languages
 - (machine independent) imperative languages
- ❑ Fourth - Non-procedural Languages
 - domain specific application generators
- ❑ Fifth - Natural Languages

Each generation is at a higher level of abstraction

How do Programming Languages Differ?

Common Constructs:

- basic data types (numbers, etc.);
- variables;
- expressions;
- statements;
- keywords;
- control constructs;
- procedures;
- comments;
- errors ...

Uncommon Constructs:

- type declarations;
 - special types (strings, arrays, matrices,...);
 - sequential execution;
 - concurrency constructs;
 - packages/modules;
 - objects;
 - general functions;
 - generics;
 - modifiable state;...
-

Language Styles ...

Procedural Languages

- Individual statements
- FORTRAN, ALGOL60, ALGOL68, Cobol, Pascal, C, Ada

Functional Languages

- When you tell the computer to do something it does it
- LISP, Scheme, CLOS, ML, Haskell

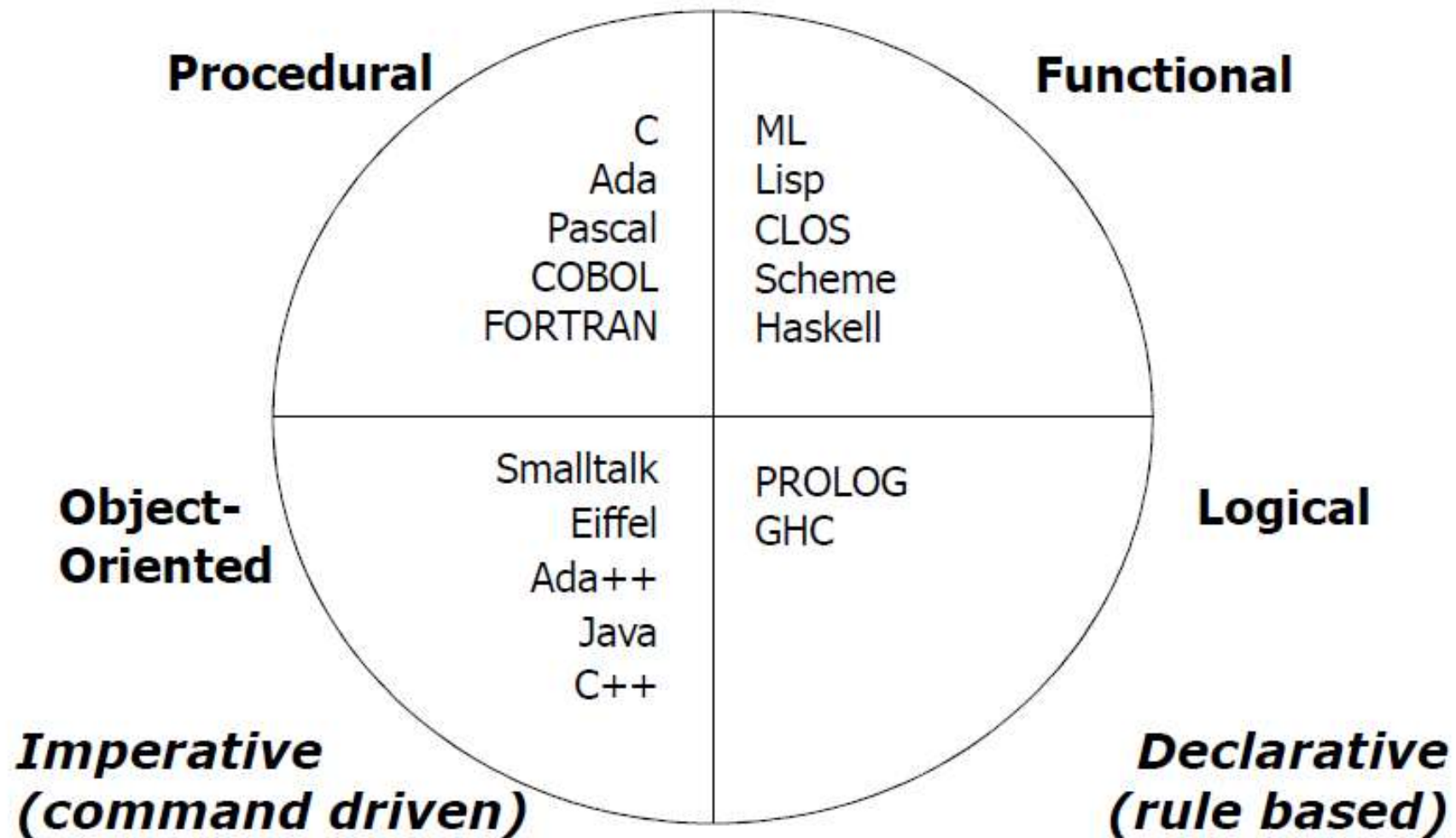
Logic Languages

- Inference engine that drives things
- Prolog, GHC

Object-oriented Languages

- Bring together data and operations
 - Smalltalk, C++, Eiffel, Sather, Python, Ada95, Java, OCAML
-

... and Programming Paradigms



Programming Paradigm

The basic structuring of thought underlying the programming activity

- ❑ Programming paradigm is a pattern or model of programming that derives the process of programming
 - ❑ Every high level language has a paradigm that guides in a problem solving within a framework and gives a solution
 - It does not mean that all high level language are strictly following one particular programming paradigm
 - ❑ Every Programming paradigm is a collection of conceptual patterns that control human thinking process to formulate the solution to a problem
 - ❑ Different programming paradigms lead to different programming techniques
-

Programming Paradigm Contd...

Four main programming paradigms –

- ❑ Imperative – Program as a collection of statements and procedures affecting data (variables). FORTRAN, BASIC, COBOL, Pascal, C
 - ❑ Functional – Program as a collection of mathematical functions. LISP, ML, Haskell
 - ❑ Logic – Program as a set of logical sentences. Prolog
 - ❑ Object Oriented – Program as a collection of classes for interacting objects. SmallTalk, C++, Java
-

Brief on various Programming Paradigms

Imperative Programming

- ❑ Idea is - “First do this and next do that”, i.e. a step-by-step execution model - based on the stored program concept of Von Neumann
 - ❑ Latin word "imperare" means “to command” - based on commands that update variables in storage
 - ❑ It describes computation in terms of statements that change a program state.
 - ❑ Similar to descriptions of everyday routines, such as food recipes
 - ❑ Natural abstraction is function, procedures or subroutines
-

Imperative Programming

(Example)

Sum of N positive numbers

```
Procedure sum(n)
Begin
    Define variable x with initial value of 1
    While the variable is greater than 1 do
        Begin
            Add x by n to store result in x
            Decrement n
        End while
    Return value of variable x
end
```

Equivalent program in C

```
int sum(unsigned int n)
{
    int x = 1;
    while(n>1)
    {
        x += n;
        n--;
    }
    return x;
}
```

What Makes a Language Imperative?

- ❑ Programs written in imperative programming languages consist of
 - A program state
 - Instructions that change the program state
 - ❑ Program instructions are “imperative” in the grammatical sense of imperative verbs that express a command
-

Concept of State

- ❑ In imperative programming, a name may be assigned to a value and later reassigned to another value.
- ❑ The collection of names and the associated values and the location of control in the program constitute the state.
- ❑ The state is a logical model of storage which is an association between memory locations and values.
- ❑ A program in execution generates a sequence of states.
- ❑ The transition from one state to the next is determined by assignment operations and sequencing commands

$$S_0 - O_0 \rightarrow S_1 - O_1 \rightarrow S_2 \dots S_{n-1} - O_{n-1} \rightarrow S_n$$

Defining Characteristics of Imperative Languages

- ❑ Statements are commands
 - Command order is critical to correct execution;
 - Programmers control all aspects: algorithm specification, memory management, variable declarations, etc.
 - ❑ They work by modifying program state
 - ❑ Statements reflect machine language instructions.
-

Structured Programming Technique

- ❑ A disciplined approach to imperative program design.
 - ❑ Uses procedural abstraction and top-down design to identify program components (also called modules or structures)
 - ❑ Program structures combined in a limited number of ways, so understanding how each structure works means you understand how the program works
 - ❑ Program control flow is based on decisions, sequences, loops, but...
 - ❑ Does not use goto statements
 - ❑ Modules are developed, tested separately and then integrated into the whole program.
-

Functional Programming

- ❑ **Functional programming** is a programming paradigm where computation is treated as the evaluation of mathematical functions, and programs are built by **composing functions**, rather than executing sequences of commands.
 - ❑ Functions are the fundamental building blocks (first class value) of a program. Functions in this sense (not to be confused with C Language functions which are just procedures) are analogous to mathematical equations: they declare a relationship between two or more entities.
 - Based on mathematical model of function composition – Lamda calculus.
-

Functional Programming

- ❑ Functional programming is a programming paradigm that treats computation as the evaluation of mathematical functions and avoids state and mutable data

In other words

- ❑ Functional programming promotes code with **no side effects, no change of value in variables**. It opposes to imperative programming, which emphasizes change of state

What it means ?

- No mutable data (no side effect)
- No state (no implicit, hidden state)

Once assigned (value binding), a variable (a symbol) does not change its value.

Key Ideas of Functional Programming

❑ Pure function:

- Always returns the same output for the same input
- Has no side effects
- No global variables, No modification of input, No state changes

❑ Functions as First-Class Values

- Functions can:
 - ✓ Be passed as arguments
 - ✓ Be returned from other functions

❑ Recursion Instead of Loops

- Functional languages often use recursion instead of for or while loops.
-

Example

Summing integers 1 to 10 in a pure functional language

**sum (m, n) : if (m > n) 0
 else m + sum (m+1, n)**

sum (1, 10) // main function

No side effect =>

No assignments to variables!

Equivalent LISP program

```
;; Define sum function
(define (sum m n)
  ( if (> m n)
        0
        (+ m (sum (+ m 1) n))
  )
)

;; Main call
(sum 1 10)
```

Example

Contd

```
Factorial(n)    = 1                if n=0  
                = n*Factorial(n-1) if n>0
```

```
(define (factorial n) (if (= n 0) 1 (* n (factorial (- n 1)))))
```

```
Fibonacci(n) = 0                if n=0  
              = 1                if n=1  
              = Fibonacci(n-1)+Fibonacci(n-2) if n>=2
```

```
(define (fib n) (cond ((= n 0) 0)  
                      ((= n 1) 1)  
                      (else (+ (fib (- n 1)) (fib (- n 2))))))
```

Referential Transparency

- ❑ A function's result depends only upon the values of its arguments and not on any previous computation or the order of evaluation for its arguments.

Or

- ❑ If we call a function with the same parameters, we know for sure the output will be the same (there is no state anywhere that would change it).
-

Referential Transparency

This is in contrast to the view of functions in imperative languages, where function values are based on arguments, order of evaluation, and can also have side effects (change state).

- Pure functional languages have no state in the sense of state in imperative programs & so no side effects.

Referential Transparency

In imperative programming a statement such as

$x = x + y;$

means “add the value of in memory cell x to the value in memory cell y and replace the value of x by the value of the function”

➤ The name x is used in two different ways here;
lhs (a memory address) versus *rhs* (a value)

❑ Functional programs accept and return data, but have no side effects and do not have the concept of a variable as a memory address

Advantages enabled by Referential Transparency

❑ Memoization

- Cache results for previous function calls.

❑ Idempotence

- Same results regardless how many times you call a function.

❑ Modularization

- We have no state that pervades the whole code, so we build our project with small, black boxes that we tie together, so it promotes bottom-up programming.
-

Advantages enabled by Referential Transparency

❑ Ease of debugging

- Functions are isolated, they only depend on their input and their output, so they are very easy to debug.

❑ Parallelization

- Functions calls are independent.
- We can parallelize in different CPUs/process/computers/...

result = func1(a, b) + func2(a, c)

We can execute *func1* and *func2* in parallel because 'a' won't be modified.

❑ Concurrency

- With no shared data, concurrency gets a lot simpler:

No semaphores, No monitors, No locks, No race-conditions, No dead-locks.

Logic Programming

- ❑ It is based on the idea of answering a question through search for solution from a knowledge base
 - ❑ Based on
 - Axioms/Facts
 - Inferences rules
 - Queries / Goals
 - ❑ Program execution becomes a systematic search in a set of facts, making use of a set of inference rules – Set of known facts and set of rules results in deduction of other facts.
 - ❑ Evaluation starts with a goal and attempts to prove it with a known fact or by deducing it from some rules.
-

Logic Programming

(Example)

Axioms/Facts

```
F1. father(dasarath, ram).  
F2. father(ram, lav).  
F3. mother(kaushalya, ram).  
F4. mother(sita, lav).
```

Inferences rules

```
R1. parent (X, Y) :- father (X, Y)  
R2. parent (X, Y) :- mother (X, Y)  
R3. grandfather(X, Y) :- father(X, Z),  
                             parent (Z, Y)  
R4. grandmother(X, Y) :- mother(X, Z),  
                             parent (Z, Y)
```

Queries / Goals

```
G1. ? father(X, ram). X = dasaratha.  
G2. ? father (dasaratha, X). X = ram  
G3. ? grandmother(X, lav). X = kaushalya.  
G4. ? parent (X, ram). X = ???
```